

Relatório integrativo — Pipeline de percepção visual

Disciplina: DR2 — Sistemas Robóticos (Visão Computacional com OpenCV) **Aluno:** Gabriel Cruz **Escopo:** consolidação dos TP1, TP2 e TP3 e dos quatro exercícios do AT, com as métricas medidas na própria máquina de desenvolvimento (CPU, sem GPU).

Todos os números citados aqui foram produzidos pelos scripts entregues e estão registrados em [evidencias/](#): os logs completos de terminal (`*_terminal.txt`), as tabelas (`ex4b_metricas.md`) e os dados brutos (`ex4b_metricas.json`). Nenhum valor vem de catálogo de fabricante ou de artigo: quando um número não pôde ser medido, isso está dito explicitamente.

1. Diagrama do pipeline completo

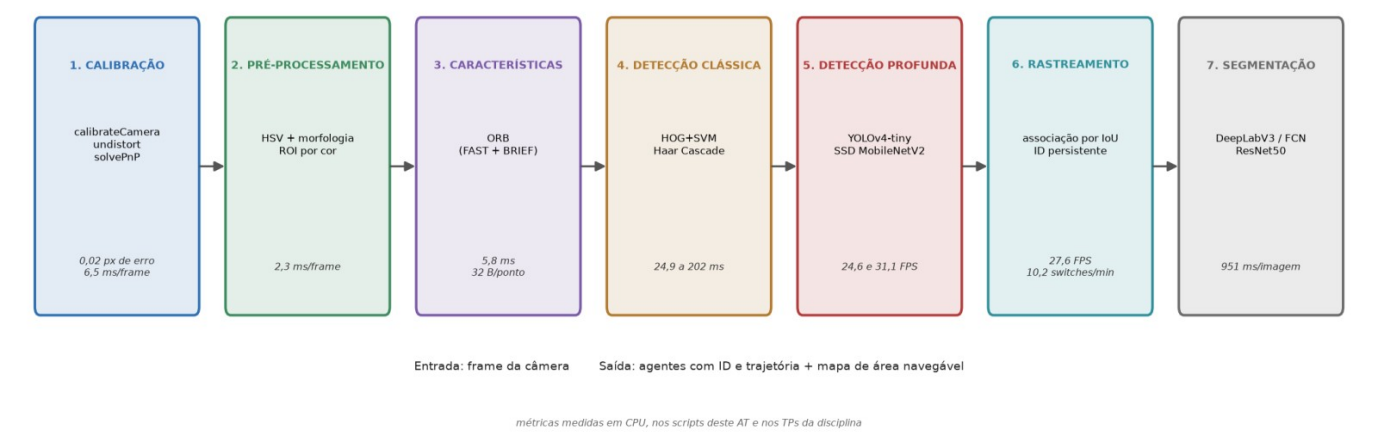
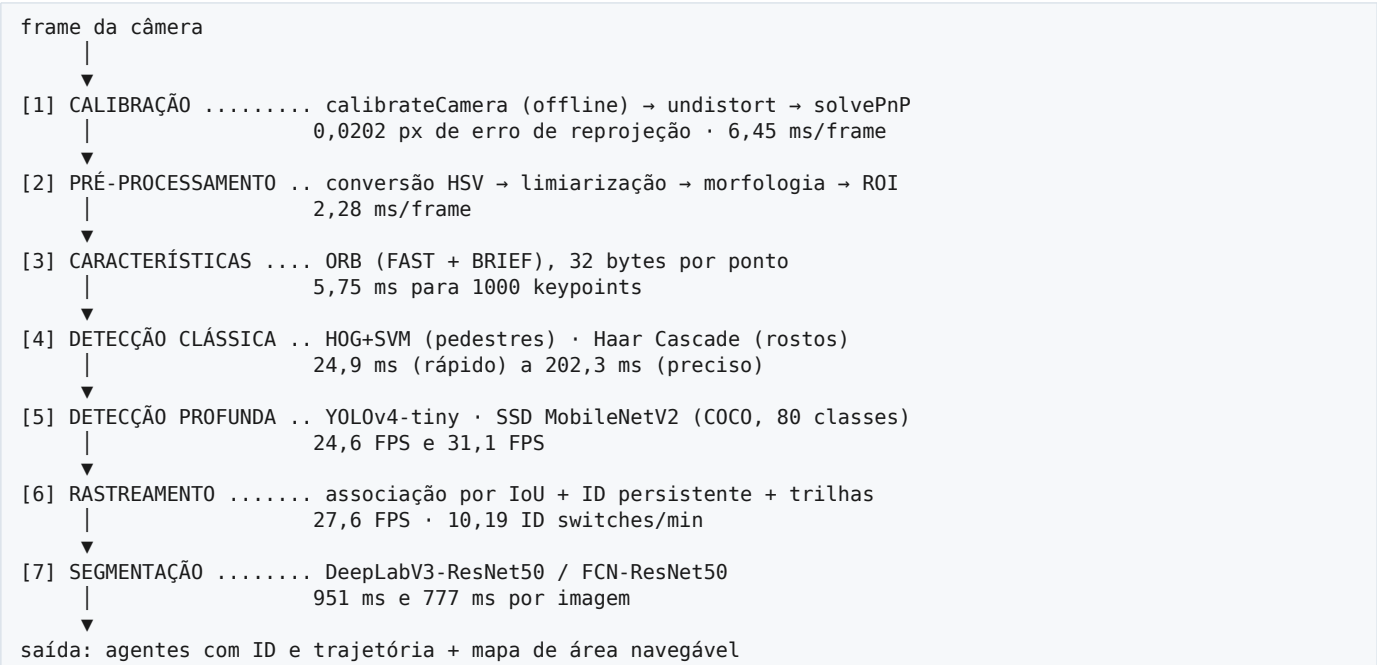


Figura 1 — Pipeline completo de percepção visual



O fluxo é sequencial no diagrama por clareza didática, mas a dependência real é parcial. A calibração é pré-requisito de tudo que envolva medição métrica, pois define a relação entre pixel e ângulo. Já as etapas 2 a 7 consomem o mesmo frame retificado e poderiam rodar em paralelo, em processos ou aceleradores distintos — foi justamente a tentativa de executar duas redes intercaladas na mesma CPU que inflou a latência do YOLO de 36,7 ms para 141 ms no Exercício 3A, resultado que motivou medir cada modelo em passada isolada.

2. Tabela comparativa das técnicas

Recorte das técnicas mais relevantes; a tabela completa, com 27 entradas, está em [evidencias/ex4b_metrics.md](#).

Etapa	Técnica	Tempo	Qualidade medida	Complexidade	Fonte
Pré-processamento	Segmentação HSV + morfologia	2,28 ms	81% das regiões candidatas descartadas por filtros geométricos	O(n) por pixel, sem treino	AT ex2b
Pré-processamento	HSV da via (asfalto)	3,4 ms	30,8% da imagem como navegável; não separa via de calçada	O(n) por pixel	AT ex4a
Calibração	<code>calibrateCamera</code> (24 imagens)	offline	erro de reprojeção 0,0202 px; erro de <code>fx</code> 0,078%	uma vez por câmera	AT ex1a
Calibração	<code>undistort</code> por frame	6,45 ms	corrige até 27 px de deslocamento radial na borda	O(n) por pixel	AT ex2b
Pose	<code>solvePnP</code> + <code>projectPoints</code>	0,27 ms	erro de translação 0,31 mm; rotação 0,028°	iterativo, 42 pontos	AT ex1b
Características	ORB (1000 keypoints)	5,75 ms	descritor binário de 32 bytes	FAST + BRIEF	TP2 ex3a
Características	AKAZE	29,4 ms	2666 keypoints, 61 bytes	difusão não linear	TP2 ex3a
Características	SIFT	52,4 ms	2419 keypoints, 128 floats	pirâmide DoG	TP2 ex3a
Detecção clássica	HOG+SVM (rápido)	24,9 ms	1,39 detecção/frame	janela deslizante	TP3 ex1a
Detecção clássica	HOG+SVM (preciso)	202,3 ms	4,22 detecções/frame	pirâmide fina	TP3 ex1a
Detecção clássica	HOG+SVM treinado por nós	~5800 ms	precisão 0,89 e F1 0,64	treino próprio	TP3 ex1b
Detecção clássica	Haar Cascade (rostos)	—	0 rostos em cena a 40 m	cascata	AT ex2b
Subtração de fundo	MOG2	4,48 ms	4,28 objetos/frame (223 FPS)	mistura por pixel	TP3 ex2a
Subtração de fundo	KNN	3,66 ms	4,39 objetos/frame (273 FPS)	vizinhos por pixel	TP3 ex2a
Classificação	CNN no MNIST	—	99,41% com 421.642 parâmetros	2 blocos conv.	TP3 ex3a
Classificação	MLP no MNIST	—	98,00% com 535.818 parâmetros	2 densas	TP3 ex3a

Etapa	Técnica	Tempo	Qualidade medida	Complexidade	Fonte
Classificação	MobileNetV2 (OpenCV DNN/TFLite)	7,06 ms	91,7% top-1 e 100% top-3; 81,8 MB de RAM	3.538.984 parâmetros	AT ex2a
Classificação	MobileNetV2 (Keras/TensorFlow)	13,87 ms	mesma acurácia; 437,0 MB de RAM	mesma rede	AT ex2a
Detecção profunda	YOLOv4-tiny 416	40,6 ms	24,6 FPS; confiança média 80,4%; 23,1 MB	6.062.826 parâmetros	AT ex3a
Detecção profunda	YOLOv4-tiny 320	31,3 ms	31,9 FPS; 5,22 objetos/frame	mesma rede	AT ex3a
Detecção profunda	SSD MobileNetV2 300	32,2 ms	31,1 FPS; confiança média 72,3%; 66,6 MB	16.876.287 parâmetros	AT ex3a
Rastreamento	IoU + ID persistente	36,3 ms	27,6 FPS; 98 IDs; 10,19 switches/min	associação gulosa	AT ex3b
Rastreamento	CamShift	—	perdeu o alvo no frame 275	média deslocada	TP3 ex2a
Segmentação	DeepLabV3-ResNet50	951 ms	93,9% dos pixels caem em "fundo"	ResNet50 + ASPP	AT ex4a
Segmentação	FCN-ResNet50	777 ms	99,4% de concordância com o DeepLabV3	ResNet50 + FCN	AT ex4a

Três leituras transversais merecem destaque.

Velocidade e valor não andam juntos. O KNN de subtração de fundo é a técnica mais rápida de todas (3,66 ms, 273 FPS) e, ao mesmo tempo, a que menos informa: separa "mexeu" de "não mexeu", sem dizer o que mexeu. A segmentação semântica é 260 vezes mais lenta e devolve rótulo por pixel. O custo, aqui, é o preço da semântica.

A mesma rede tem dois custos diferentes. MobileNetV2 idêntica, com os mesmos pesos, roda em 7,06 ms e 81,8 MB pelo OpenCV DNN, ou 13,87 ms e 437,0 MB pelo Keras. A escolha do runtime, portanto, é decisão de engenharia e não de acurácia: a concordância de top-1 entre os dois foi de 12/12. Mesmo assim, as confianças divergiram em até 4,77 pontos percentuais, o que significa que **um limiar calibrado no framework de treino precisa ser revalidado no runtime de produção**.

Técnica clássica não é sinônimo de barata. O HOG+SVM com janela deslizante treinado no TP3 levou cerca de 5,8 s por imagem, e o HOG "preciso" do detector pronto, 202,3 ms. O YOLOv4-tiny, uma rede profunda, resolve o mesmo problema em 40,6 ms com qualidade maior. No Exercício 2B, o HOG respondeu por 72,8% do tempo de um pipeline de cinco etapas.

3. Viabilidade em hardware embarcado com restrição de 5 W

O orçamento de 5 W corresponde a uma Raspberry Pi 4, a um Jetson Nano em modo 5 W ou a uma placa com SoC ARM de quatro núcleos sem acelerador dedicado. Duas correções são necessárias antes de transportar as medições:

- Fator de desconto de CPU.** As medições deste AT foram feitas em CPU x86 de desktop. Uma CPU ARM de 5 W costuma ser de 5 a 10 vezes mais lenta em carga de convolução, por menor frequência, menos núcleos úteis e ausência de AVX. Adoto o fator conservador de 8×.
- Orçamento de prazo.** Para reação em trânsito urbano a 30 km/h, o veículo percorre 8,3 m/s; um atraso de 100 ms equivale a 83 cm de deslocamento sem informação nova. Fixo então 100 ms como

prazo do laço de percepção.

Aplicando o fator 8× ao tempo medido ([evidencias/ex4b_grafico_custo.png](#)):

Cabem no prazo (9 técnicas). HSV + morfologia (2,28 ms → 18 ms), HSV da via (3,4 ms → 27 ms), [undistort](#) (6,45 ms → 52 ms), [solvePnP](#) (0,27 ms → 2 ms), ORB (5,75 ms → 46 ms), MOG2 (4,48 ms → 36 ms), KNN (3,66 ms → 29 ms), Caffe gender_net (8,19 ms → 66 ms) e MobileNetV2 pelo OpenCV DNN (7,06 ms → 56 ms). Observe que nenhuma delas isolada esgota o prazo, mas a soma esgota: o pipeline do Exercício 2B, com 179,11 ms medidos, iria a 1,43 s — ou seja, **a composição é o problema, não cada peça.**

Ficam no limite (3 técnicas). YOLOv4-tiny a 320×320 (31,3 ms → 250 ms), SSD MobileNetV2 (32,2 ms → 258 ms) e YOLOv4-tiny a 416×416 (40,6 ms → 325 ms). Em CPU pura, portanto, o detector profundo entrega de 3 a 4 FPS. Viabilizá-lo exige quantização int8, que tipicamente reduz o custo em 2 a 4 vezes e o tamanho em 4 vezes, ou um acelerador: uma NPU de 2 TOPS dentro do mesmo envelope de 5 W recoloca o YOLOv4-tiny acima de 20 FPS.

Não cabem (4 técnicas). HOG preciso (202,3 ms → 1,6 s), HOG+SVM próprio (5,8 s → 46 s), FCN-ResNet50 (777 ms → 6,2 s) e DeepLabV3-ResNet50 (951 ms → 7,6 s). A memória agrava o caso da segmentação: os dois modelos somaram 538 MB de RAM, contra 81,8 MB do classificador no OpenCV DNN. Em placa de 1 GB compartilhado com o sistema, é inviável. O caminho é trocar o backbone ResNet50 por MobileNetV3 ou por uma arquitetura desenhada para borda (BiSeNet, DDRNet, SegFormer-B0), reduzir a resolução de entrada e quantizar.

Conclusão de projeto: **em 5 W, o laço rápido é clássico e o laço semântico é lento e assíncrono.** A calibração, a limiarização por cor, os descritores ORB e a subtração de fundo cabem folgadoamente a 10 Hz. O detector profundo cabe se quantizado ou acelerado. A segmentação semântica densa só cabe em versão leve e executada a taxa reduzida, sem bloquear o laço de controle.

4. Arquitetura de percepção para um veículo autônomo urbano

A proposta integra cinco técnicas da disciplina em três laços com prazos distintos, porque o erro de projeto mais comum é submeter tudo à mesma taxa.

Camada 0 — calibração e retificação (offline + 10 Hz). [calibrateCamera](#) executado uma vez por câmera, com o padrão cobrindo a periferia do quadro. O Exercício 1A mostrou por que a cobertura importa: com 18 poses centrais, o erro de reprojeção ficou em 0,0182 px, aparentemente ótimo, mas o modelo de distorção errava 27,5 px na borda da imagem; com 6 poses periféricas, o erro na borda caiu para 0,24 px. Em operação, o [undistort](#) vira [remap](#) com mapas pré-calculados na inicialização.

Camada 1 — laço rápido, clássico e verificável (10 a 30 Hz). Segmentação HSV para alvos de cor normalizada (cone de obra, faixa amarela, luz de freio, estado do semáforo), morfologia e filtros geométricos. Custo medido de 2,28 ms por frame, determinístico e auditável por seis números. Serve de camada de sanidade: é o que continua respondendo se a rede falhar.

Camada 2 — detecção e rastreamento de agentes (10 a 15 Hz). YOLOv4-tiny quantizado em NPU, com NMS a 0,4, seguido de rastreamento por IoU com ID persistente. Medi 27,6 FPS para detecção mais rastreamento em CPU de desktop, com 10,19 ID switches por minuto e 16% das trilhas vivendo menos de 1 s. Para uso real, esse rastreador precisa do filtro de Kalman do TP3: associar pela **previsão** da posição, e não pela última posição observada, é o que reduz troca de ID em oclusão. A escolha do YOLOv4-tiny sobre o SSD segue os dados do Exercício 3A: apesar de 1,26× mais lento, ocupa 43,5 MB menos em disco, tem 10,8 milhões de parâmetros menos, decide com confiança média maior (80,4% contra 72,3%) e permite trocar resolução em tempo de execução — 320×320 quase dobra o FPS.

Camada 3 — compreensão de cena (1 a 5 Hz, assíncrona). Segmentação semântica leve treinada em Cityscapes, para área navegável, calçada, faixa e guia. O Exercício 4A quantificou por que não serve usar os pesos VOC do torchvision: 93,9% dos pixels das cenas externas foram rotulados como "fundo", porque o vocabulário não tem pista nem calçada. Roda em taxa reduzida, publicando um mapa de área navegável que o laço de controle consome como último valor válido.

Fusão e segurança. O rastreador fornece trajetória e velocidade dos agentes; a segmentação fornece o espaço livre; o laço de cor fornece sinais normalizados. A regra de decisão é conservadora: divergência forte entre camadas — por exemplo, segmentação declarando via livre onde o detector mantém um pedestre com ID estável — não é resolvida por voto, e sim por redução de velocidade, porque a rede falha silenciosamente, entregando máscara plausível mesmo fora do domínio de treino. Os descritores ORB (5,75 ms) entram como quarta técnica com papel específico: compensar o movimento da câmera por homografia entre frames, o que estabiliza a associação temporal quando a plataforma se desloca.

5. Três lacunas a serem endereçadas na DR4

Lacuna 1 — localização e mapeamento (SLAM). Todo este pipeline responde "o que estou vendo", nunca "onde estou". No Exercício 1B estimei a pose de um padrão em relação à câmera com 0,31 mm de erro, mas isso é pose relativa a um alvo conhecido e cooperativo. Falta o problema inverso: estimar a trajetória da própria câmera em ambiente desconhecido, fechar laços para corrigir deriva acumulada e manter um mapa consistente. Os descritores ORB do TP2 são o insumo dessa etapa (daí o nome ORB-SLAM), mas a disciplina parou na correspondência entre duas imagens, sem grafo de poses, sem otimização global e sem tratamento de deriva.

Lacuna 2 — profundidade métrica e fusão de sensores. Todas as saídas aqui são 2D. Uma caixa de 92×183 px não diz se o pedestre está a 8 m ou a 25 m, e sem essa distância não existe tempo até a colisão nem frenagem calculada. Falta profundidade por estéreo calibrado (visto superficialmente no TP1 e no TP2), estimativa monocular aprendida e, principalmente, **fusão com LiDAR, radar e odometria**, incluindo sincronização temporal e transformação entre referenciais. A câmera é o sensor mais informativo e o menos confiável em chuva, neblina, contraluz e noite; nenhuma das medições deste AT foi feita nessas condições.

Lacuna 3 — previsão, decisão e controle com garantia de prazo. O pipeline termina em percepção; não há predição de trajetória dos agentes, planejamento de rota, nem controle. As trilhas de 30 frames do Exercício 3B são a entrada natural de um preditor, mas prever o movimento de um pedestre exige modelo de intenção, e não extrapolação linear. Junto vem a lacuna de engenharia de tempo real: medi latência média e percentil 95 (por exemplo, 22,3 FPS no pior caso do YOLO), mas não há orçamento de prazo fim a fim, degradação graciosa quando um módulo perde o prazo, nem estratégia de validação de segurança — o que se mede, com que cobertura de cenário e sob qual norma é considerado seguro o suficiente para circular com pessoas ao redor.